

Nama : Khairul Insan
NIM : 2410511018
Kelas : A
Studi kasus : Sistem manajemen kos atau sewa properti
Provider OAuth : Google

LAPORAN UTS PEMBANGUNAN PERANGKAT LUNAK BERORIENTASI SERVICE

1. Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi perangkat lunak menuntut sistem yang scalable, fleksibel, dan mudah dikembangkan. Pendekatan monolitik yang menggabungkan seluruh fitur dalam satu aplikasi seringkali menyulitkan dalam hal maintenance, deployment, dan scalability.

Oleh karena itu, arsitektur service oriented atau microservices menjadi solusi yang lebih modern. Sistem dipecah menjadi beberapa layanan kecil yang saling terintegrasi melalui API, dibangun sebuah sistem manajemen kos atau sewa properti yang meliputi pengelolaan properti, booking, pembayaran, serta autentikasi pengguna menggunakan JWT dan OAuth 2.0.

1.2 Tujuan

Tujuan dari pembangunan sistem ini adalah:

- Menerapkan arsitektur microservices dalam pengembangan aplikasi
- Mengimplementasikan REST API dengan standar yang benar
- Menerapkan autentikasi menggunakan JWT
- Mengintegrasikan OAuth 2.0 (Google Login)
- Membangun API Gateway sebagai single entry point
- Memahami komunikasi antar service

2. Arsitektur Sistem

Sistem ini dibangun menggunakan arsitektur microservices yang terdiri dari beberapa layanan terpisah yang saling berkomunikasi melalui REST API.

2.1 Komponen Sistem

- API Gateway: Berfungsi sebagai pintu utama untuk semua request dari client. Gateway menangani routing ke service terkait, validasi JWT, dan rate limiting.
- Auth Service: Mengelola autentikasi pengguna, termasuk login menggunakan email atau password dan OAuth Google, serta pengelolaan JWT dan refresh token.
- Property Service: Mengelola data properti, kamar, dan fasilitas. Service ini dibangun menggunakan Laravel (PHP MVC).
- Booking Service: Menangani proses pemesanan kamar oleh pengguna.
- Payment Service: Mengelola proses pembayaran dan riwayat transaksi.

2.2 Alur Sistem

Client mengirim request ke API Gateway, kemudian gateway meneruskan request ke service yang sesuai. Setiap service memiliki database masing masing dan tidak saling berbagi data secara langsung.

3. Justifikasi Microservice

Pemecahan sistem menjadi beberapa service dilakukan dengan pertimbangan sebagai berikut:

- Auth Service dipisahkan karena autentikasi dapat digunakan oleh semua service dan memerlukan keamanan khusus.
- Property Service dipisahkan karena memiliki kompleksitas data relasional yang tinggi.
- Booking Service dipisahkan karena memiliki logika bisnis transaksi yang berbeda.
- Payment Service dipisahkan karena berhubungan dengan transaksi keuangan dan riwayat pembayaran.

Perbandingan dengan Monolitik

Pada arsitektur monolitik semua fitur berada dalam satu aplikasi sehingga sulit untuk dikembangkan secara paralel dan sulit diskalakan. Sedangkan pada microservices:

- Setiap service dapat dikembangkan secara independen
- Lebih mudah melakukan scaling pada service tertentu
- Lebih mudah dalam maintenance

4. Implementasi Rest API

Sistem ini menggunakan REST API dengan penggunaan HTTP method yang sesuai:

- GET: mengambil data
- POST: membuat data baru
- PUT atau PATCH: mengupdate data
- DELETE: menghapus data

Response Code yang digunakan:

- 200: sukses
- 201: data berhasil dibuat
- 204: sukses tanpa response body
- 400: bad request
- 401: unauthorized
- 403: forbidden
- 404: data tidak ditemukan
- 409: conflict
- 422: validasi gagal
- 500: server error

5. JWT dan Oauth

5.1 JWT (JSON Web Token)

JWT digunakan sebagai mekanisme autentikasi utama. Setelah login sistem menghasilkan:

- Access Token (berlaku 15 menit)
- Refresh Token (berlaku 7 hari)

Access token digunakan untuk mengakses endpoint yang dilindungi, sedangkan refresh token digunakan untuk mendapatkan access token baru tanpa login ulang.

5.2 OAuth 2.0 (Google Login)

Sistem juga menyediakan login menggunakan Google dengan menggunakan flow Authorization Code.

Alur:

1. User login melalui Google
2. Google mengirimkan data user ke sistem
3. Sistem menyimpan data user
4. Sistem menghasilkan JWT

Data yang disimpan meliputi:

- Nama
- Email
- Foto profil

6. API Gateway

API Gateway berfungsi sebagai penghubung antara client dan service backend.

Fungsi utama:

- Routing: meneruskan request ke service tujuan
- JWT Validation: memvalidasi token sebelum request diproses
- Rate Limiting: membatasi request

Semua request dari client wajib melalui gateway

7. Database

Setiap service memiliki database sendiri, yaitu:

- Auth Service: auth_db
- Property Service: property_db
- Booking Service: booking_db
- Payment Service: payment_db

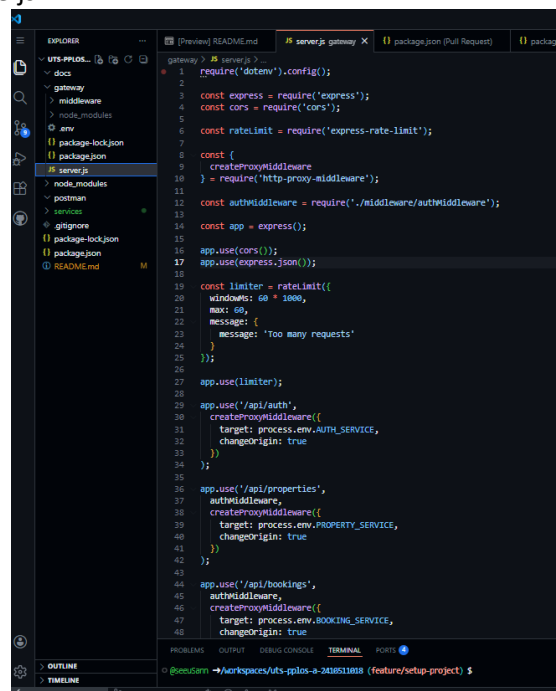
Pendekatan ini digunakan untuk menjaga independensi antar service dan menghindari ketergantungan langsung antar database.

8. Kesimpulan

Sistem manajemen kos berbasis microservice memanfaatkan berbagai teknologi seperti Node.js, Laravel, JWT, dan OAuth 2.0. Dengan pendekatan sistem menjadi lebih modular, scalable, dan mudah dikembangkan dibandingkan arsitektur monolitik. Implementasi API Gateway, JWT, dan OAuth juga meningkatkan keamanan serta fleksibilitas dalam autentikasi pengguna.

9. Daftar Gambar

- Dokumentasi Node.js



```
1 require('dotenv').config();
2
3 const express = require('express');
4 const cors = require('cors');
5
6 const rateLimit = require('express-rate-limit');
7
8 const {
9   createProxyMiddleware
10 } = require('http-proxy-middleware');
11
12 const authMiddleware = require('./middleware/authMiddleware');
13
14 const app = express();
15
16 app.use(cors());
17 app.use(express.json());
18
19 const limiter = rateLimit({
20   windowMs: 60 * 1000,
21   max: 60,
22   message: {
23     message: 'Too many requests'
24   }
25 });
26
27 app.use(limiter);
28
29 app.use('/api/auth',
30   createProxyMiddleware({
31     target: process.env.AUTH_SERVICE,
32     changeOrigin: true
33   })
34 );
35
36 app.use('/api/properties',
37   authMiddleware,
38   createProxyMiddleware({
39     target: process.env.PROPERTY_SERVICE,
40     changeOrigin: true
41   })
42 );
43
44 app.use('/api/bookings',
45   authMiddleware,
46   createProxyMiddleware({
47     target: process.env.BOOKING_SERVICE,
48     changeOrigin: true
49   })
50 );
```

The Explorer sidebar shows the project structure with the following files and folders:

- UTS-PPLOS-A-2410511018 (CO...)
 - docs
 - gateway
 - middleware
 - node_modules
 - .env
 - package-lock.json
 - package.json
 - server.js
 - node_modules
 - postman
 - services
 - auth-service
 - config
 - controllers
 - middleware
 - node_modules
 - routes
 - .env
 - package-lock.json
 - package.json
 - booking-service
 - payment-service
 - property-service
 - gitignore
 - package-lock.json
 - package.json
 - README.md

The Editor shows the `server.js` file for the `auth-service` directory:

```
1 require('dotenv').config();
2
3 const express = require('express');
4 const cors = require('cors');
5
6 const app = express();
7
8 app.use(cors());
9 app.use(express.json());
10
11 const authRoutes = require('./routes/authRoutes');
12
13 app.use('/', authRoutes);
14
15 const passport = require('passport');
16 require('./config/passport');
17
18 app.use(passport.initialize());
19
20 app.listen(process.env.PORT, () => {
21   console.log('Auth service running on ${process.env.PORT}');
22 });
```

The Explorer sidebar shows the project structure with the following files and folders:

- UTS-PPLOS-A-2410511018 (CO...)
 - docs
 - gateway
 - middleware
 - node_modules
 - .env
 - package-lock.json
 - package.json
 - server.js
 - node_modules
 - postman
 - services
 - auth-service
 - config
 - controllers
 - middleware
 - node_modules
 - routes
 - .env
 - package-lock.json
 - package.json
 - booking-service
 - controllers
 - node_modules
 - routes
 - .env
 - package-lock.json
 - package.json
 - payment-service
 - property-service
- gitignore
- package-lock.json
- package.json
- README.md

The Editor shows the `server.js` file for the `booking-service` directory:

```
1 require('dotenv').config();
2
3 const express = require('express');
4 const cors = require('cors');
5
6 const bookingRoutes = require('./routes/bookingRoutes');
7
8 const app = express();
9
10 app.use(cors());
11 app.use(express.json());
12
13 app.use('/bookings', bookingRoutes);
14
15 app.listen(process.env.PORT, () => {
16   console.log('Booking service running on ${process.env.PORT}');
17 });
```

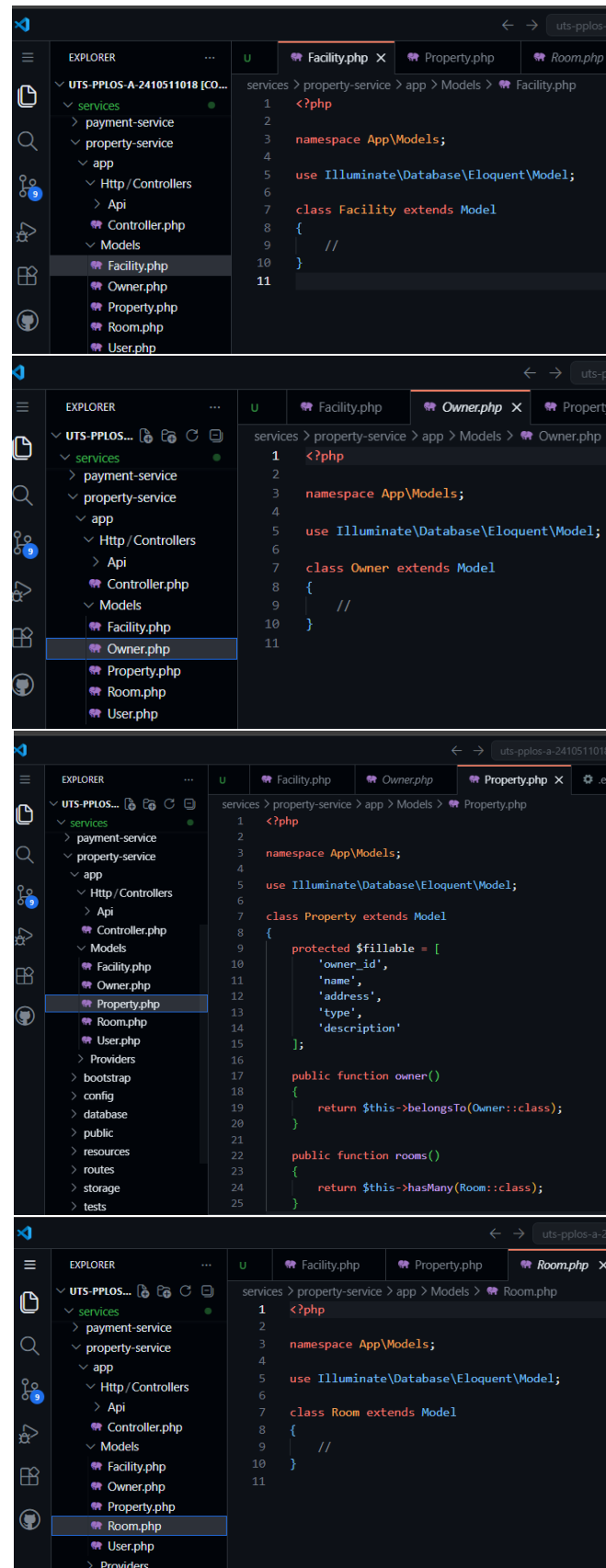
The Explorer sidebar shows the project structure with the following files and folders:

- UTS-PPLOS-A-2410511018 (CO...)
 - docs
 - gateway
 - middleware
 - node_modules
 - .env
 - package-lock.json
 - package.json
 - server.js
 - node_modules
 - postman
 - services
 - auth-service
 - config
 - controllers
 - middleware
 - node_modules
 - routes
 - .env
 - package-lock.json
 - package.json
 - booking-service
 - payment-service
 - controllers
 - node_modules
 - routes
 - .env
 - package-lock.json
 - package.json
 - property-service
- gitignore
- package-lock.json
- package.json
- README.md

The Editor shows the `server.js` file for the `payment-service` directory:

```
1 require('dotenv').config();
2
3 const express = require('express');
4 const cors = require('cors');
5
6 const paymentRoutes = require('./routes/paymentRoutes');
7
8 const app = express();
9
10 app.use(cors());
11 app.use(express.json());
12
13 app.use('/payments', paymentRoutes);
14
15 app.listen(process.env.PORT, () => {
16   console.log('Payment service running on ${process.env.PORT}');
17 });
```

- Dokumentasi Laravel



```
1 <?php
2
3 namespace App\Models;
4
5 // use Illuminate\Contracts\Auth\MustVerifyEmail;
6 use Database\Factories\UserFactory;
7 use Illuminate\Database\Eloquent\Attributes\Fillable;
8 use Illuminate\Database\Eloquent\Attributes\Hidden;
9 use Illuminate\Database\Eloquent\Factories\HasFactory;
10 use Illuminate\Foundation\Auth\User as Authenticatable;
11 use Illuminate\Notifications\Notifiable;
12
13 #[Fillable(['name', 'email', 'password'])]
14 #[Hidden(['password', 'remember_token'])]
15 class User extends Authenticatable
16 {
17     /** @use HasFactory<UserFactory> */
18     use HasFactory, Notifiable;
19
20     /**
21      * Get the attributes that should be cast.
22      *
23      * @return array<string, string>
24      */
25     protected function casts(): array
26     {
27         return [
28             'email_verified_at' => 'datetime',
29             'password' => 'hashed',
30         ];
31     }
32 }
33
```

- Dokumentasi JWT

```
1 const jwt = require('jsonwebtoken');
2
3 module.exports = (req, res, next) => {
4
5     const authHeader = req.headers.authorization;
6
7     if (!authHeader) {
8         return res.status(401).json({
9             message: 'Token required'
10         });
11     }
12
13     const token = authHeader.split(' ')[1];
14
15     try {
16
17         const decoded = jwt.verify(
18             token,
19             process.env.JWT_SECRET
20         );
21
22         req.user = decoded;
23
24         next();
25
26     } catch (error) {
27
28         return res.status(401).json({
29             message: 'Invalid token'
30         });
31     }
32 }
33
34
```

```
1 const bcrypt = require('bcrypt');
2 const jwt = require('jsonwebtoken');
3
4 const db = require('../config/db');
5
6 exports.register = async (req, res) => {
7
8   try {
9
10    const { name, email, password } = req.body;
11
12    if (!name || !email || !password) {
13      return res.status(422).json({
14        message: 'All fields required'
15      });
16    }
17
18    db.query(
19      'SELECT * FROM users WHERE email = ?',
20      [email],
21      async (err, results) => {
22
23        if (results.length > 0) {
24          return res.status(409).json({
25            message: 'Email already used'
26          });
27        }
28
29        const hashedPassword = await bcrypt.hash(password, 10);
30
31        db.query(
32          'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
33          [name, email, hashedPassword],
34          (err, result) => {
35
36            if (err) {
37              return res.status(500).json({
38                message: err.message
39              });
40            }
41
42            res.status(201).json({
43              message: 'Register success'
44            });
45          });
46        }
47      }
48    );
49  }
50 }
51
52 exports.login = async (req, res) => {
53
54   try {
55
56     const { email, password } = req.body;
57
58     db.query(
59       'SELECT * FROM users WHERE email = ?',
60       [email],
61       async (err, results) => {
62
63         if (results.length === 0) {
64           return res.status(401).json({
65             message: 'Invalid credentials'
66           });
67         }
68
69         const user = results[0];
70
71         const validPassword = await bcrypt.compare(
72           password,
73           user.password
74         );
75
76         if (!validPassword) {
77           return res.status(401).json({
78             message: 'Invalid credentials'
79           });
80         }
81
82         const accessToken = jwt.sign(
83           {
84             id: user.id,
```

```
85       id: user.id,
86     },
87     { expiresIn: '1h' }
88   );
89
90   res.status(200).json({
91     message: 'Login success',
92     accessToken
93   });
94 }
95
96 exports.refresh = async (req, res) => {
97
98   try {
99
100    const { refreshToken } = req.body;
101
102    if (!refreshToken) {
103      return res.status(401).json({
104        message: 'Refresh token required'
105      });
106    }
107
108    const decodedToken = jwt.verify(refreshToken, process.env.REFRESH_TOKEN_SECRET);
109
110    const user = await db.query(
111      'SELECT * FROM users WHERE id = ?',
112      [decodedToken.id],
113      (err, results) => {
114
115        if (err) {
116          return res.status(500).json({
117            message: err.message
118          });
119        }
120
121        if (results.length === 0) {
122          return res.status(401).json({
123            message: 'User not found'
124          });
125        }
126
127        const user = results[0];
128
129        const accessToken = jwt.sign(
130          {
131            id: user.id,
```

```
services > auth-service > controllers > .js authController.js > refresh > refresh
62 exports.login = async (req, res) => {
71   async (err, results) => {
183     const refreshToken = jwt.sign(
186       process.env.JWT_SECRET,
188       {
189         expiresIn: process.env.REFRESH_TOKEN_EXPIRES
190       }
191     );
192     db.query(
193       'INSERT INTO refresh_tokens (token, user_id) VALUES (?, ?)',
194       [refreshToken, user.id]
195     );
196     res.status(200).json({
197       accessToken,
198       refreshToken
199     });
200   } catch (error) {
201     res.status(500).json({
202       message: error.message
203     });
204   }
205 }
206 exports.refresh = (req, res) => {
207   const { refreshToken } = req.body;
208   if (!refreshToken) {
209     return res.status(401).json({
210       message: 'Refresh token required'
211     });
212   }
213   db.query(
214     'SELECT * FROM refresh_tokens WHERE token = ?'
215   );
216 }
```

```
services > auth-service > controllers > .js authController.js > refresh > refresh
136 exports.refresh = (req, res) => {
149   (err, results) => {
158     refreshToken,
159     process.env.JWT_SECRET,
160     (err, decoded) => {
161       if (err) {
162         return res.status(403).json({
163           message: 'Expired refresh token'
164         });
165       }
166       const newAccessToken = jwt.sign(
167         { id: decoded.id },
168         process.env.JWT_SECRET,
169         { expiresIn: process.env.ACCESS_TOKEN_EXPIRES }
170       );
171       res.json({
172         accessToken: newAccessToken
173       });
174     }
175   }
176 }
177 router.post('/refresh', authController.refresh);
178
179 exports.logout = (req, res) => {
180   const { refreshToken } = req.body;
181   db.query(
182     'DELETE FROM refresh_tokens WHERE token = ?',
183     [refreshToken],
184     () => {
185       res.json({
186         message: 'Logout success'
187       });
188     }
189   );
190 }
```

- Dokumentasi OAuth 2.0 Google

OAuth client created

i OAuth access is restricted to the [test users](#) listed on your [OAuth consent screen](#)

Client ID	519738388029-lilc6arsflua566k5pbfbImbed1uhbpj.apps.googleusercontent.com
-----------	--

⚠ You will no longer be able to view or download the client secret once you close this dialog. Make sure you have copied or downloaded the information below and securely stored it.

Client secret	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
Creation date	May 2, 2026, 5:32:36 PM GMT+7
Status	✔ Enabled

[Download JSON](#)

OK